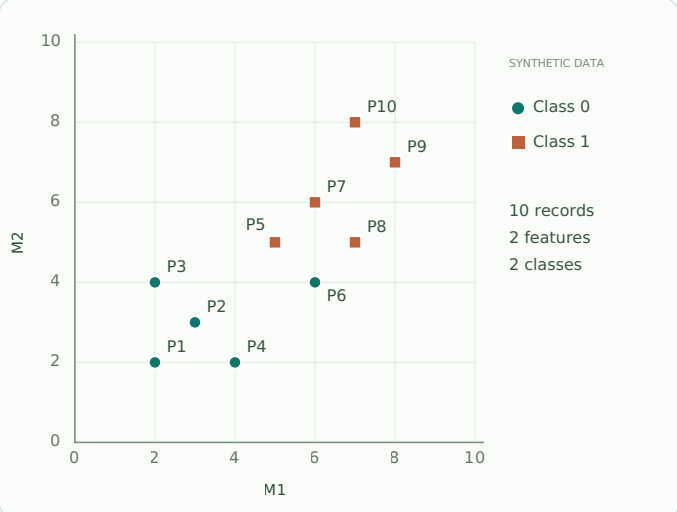
یادداشت ۰۲

از مشورت گرفتن تا طبقه‌بندیِ باینری

دو ویژگی، دو کلاس، یک پرس‌وجو

فهم kNN با دادهٔ پزشکیِ ساختگی



Instructor

Masoud Ahangary

ادامهٔ یادداشت «از یک جدول تا نزدیک‌ترین نمونه»

یادداشت ۰۲ / طبقه‌بندیِ باینری با همسایه‌ها

برای یک تصمیم تازه، سراغ چه کسی می‌رویم؟

فرض کنید می‌خواهید در یک دورهٔ آموزشی شرکت کنید و هنوز مطمئن نیستید انتخابِ مناسبی است. از کسی که قبلاً آن دوره را گذرانده می‌پرسید: «برای من هم مناسب است؟» پاسخِ او وقتی برایتان مفیدتر می‌شود که شرایطش به شما شبیه باشد؛ مثلاً پیش‌زمینه و هدفِ یادگیریِ نزدیک‌تری داشته باشد.

حالا به‌جای یک نفر، از سه نفر با شرایطِ مشابه مشورت می‌گیرید. دو نفر تجربهٔ مثبتی داشته‌اند و یک نفر نظرِ منفی دارد. اگر قرار باشد هر نفر یک رأی داشته باشد، جمع‌بندیِ این سه تجربه، نظرِ مثبت خواهد بود. با این حال، رأیِ بیشتر، درست بودنِ تصمیم را تضمین نمی‌کند؛ کیفیتِ تجربه‌ها و میزانِ شباهتِ شرایط هم اهمیت دارد.

تجربهٔ مشابه ۱	تجربهٔ مشابه ۲	تجربهٔ مشابه ۳
موافق	**مخالف**	**موافق**

برای تبدیلِ این تمثیل به الگوریتم، باید روشن کنیم چه چیزی جایِ هر بخش را می‌گیرد. شرایطِ افراد، همان ویژگی‌هاست؛ نزدیک بودنِ شرایط را با فاصله می‌سنجیم؛ تجربهٔ قبلیِ هر فرد، یک پاسخِ ثبت‌شده دارد؛ و جمع‌بندیِ چند تجربه، رأی‌گیری می‌شود. در مثالِ پزشکیِ ما، پرونده‌ها نظر نمی‌دهند؛ نتیجهٔ بررسیِ قبلیِ هر پرونده، رأیِ آن را مشخص می‌کند.

در تمثیلِ مشورت	در مدل
شرایطِ فرد	ویژگی‌های ورودی
شباهتِ شرایط	فاصله در فضای ویژگی
نتیجهٔ تجربهٔ قبلی	برچسبِ ثبت‌شده
جمع‌بندیِ چند تجربه	رأیِ اکثریتِ همسایه‌ها

در جلسهٔ قبل، بخشِ «پیدا کردنِ نمونه‌های مشابه» را ساخته‌ایم: بردارهای ویژگی را می‌خوانیم، فاصله‌ها را حساب می‌کنیم و نزدیک‌ترین‌ها را برمی‌گزینیم. این بار می‌خواهیم یک قدم جلوتر برویم و از همسایه‌های انتخاب‌شده، برای تعیینِ یک کلاس استفاده کنیم.

> در تمثیلِ ما، تعدادِ افرادی که از آن‌ها مشورت می‌گیریم، نقشِ k را دارد. در الگوریتم، هر رأی از برچسبِ ثبت‌شدهٔ یک نمونهٔ نزدیک می‌آید.

یادداشت ۰۲ / طبقه‌بندیِ باینری با همسایه‌ها

پرونده‌های قبلی، همراهِ پاسخ‌اند

یک درمانگاهِ کاملاً فرضی را تصور کنید. برای هر مراجعه‌کننده، دو امتیازِ آموزشی ثبت کرده‌ایم: M1 امتیازِ نشانه‌های تنفسی و M2 امتیازِ خستگی است. هر امتیاز بین صفر و ده قرار دارد. نتیجهٔ یک بررسیِ مرجعِ فرضی نیز از قبل ثبت شده و وجود یا نبودِ وضعیتِ S را مشخص می‌کند: صفر برای «منفی» و یک برای «مثبت».

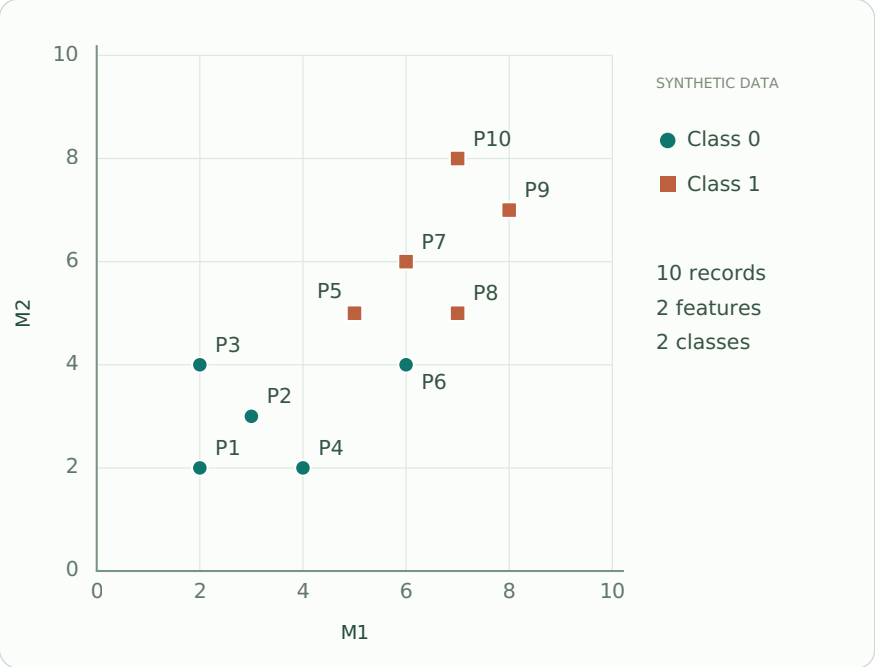
پرونده	تنفسی / M1	خستگی / M2	برچسبِ ثبت‌شده
P1	2	2	○ ۰ — منفی
P2	3	3	○ ۰ — منفی
P3	2	4	○ ۰ — منفی
P4	4	2	○ ۰ — منفی
P5	5	5	■ ۱ — مثبت
P6	6	4	○ ۰ — منفی
P7	6	6	■ ۱ — مثبت
P8	7	5	■ ۱ — مثبت
P9	8	7	■ ۱ — مثبت
P10	7	8	■ ۱ — مثبت

این امتیازدهی، پرونده‌ها و وضعیتِ S برای آموزش ابداع شده‌اند و ابزارِ بالینی نیستند. برچسبِ منفی یا مثبت، فقط پاسخِ همان بررسیِ فرضی است. برای اینکه اثرِ فاصله را ساده ببینیم، در این مثال یک واحد تغییرِ هر امتیاز را هم‌وزن می‌گیریم؛ مشترک بودنِ بازهٔ صفر تا ده، به‌خودیِ‌خود اهمیتِ واقعیِ دو ویژگی را برابر نمی‌کند.

اکنون مراجعه‌کنندهٔ تازه‌ای داریم که دو شاخصش معلوم است، اما نتیجهٔ بررسیِ مرجعِ او را در اختیار نداریم. هدفِ ما پیش‌بینیِ یکی از دو برچسب، با کمکِ پرونده‌های قبلی است. دو شاخص، **ویژگی‌های ورودی** هستند و نتیجهٔ ثبت‌شدهٔ بررسی، **متغیرِ هدف یا برچسب** است.

> این یک مسئلهٔ یادگیری نظارت‌شده است: برای نمونه‌های گذشته، هم ورودی و هم پاسخِ معلوم داریم و می‌خواهیم برای ورودیِ تازه پیش‌بینی بسازیم.

دو ویژگی روی صفحه، دو کلاس برای پاسخ



محور افقی: امتیازِ تنفسی M1؛ محور عمودی: امتیازِ خستگی M2. دایره‌ها کلاس ۰ و مربع‌ها کلاس ۱ هستند.

هر پرونده با دو عدد مشخص می‌شود، پس می‌توانیم آن را روی یک صفحه قرار دهیم. برای مثال، P5 با مختصاتِ ۵ و ۵ و P6 با مختصاتِ ۶ و ۴ رسم شده‌اند. شکل و رنگِ علامت، پاسخِ ثبت‌شدهٔ هر پرونده را نشان می‌دهند؛ مختصاتِ نقطه از دو ویژگی می‌آید.

$$y \in \{0, 1\}$$

این مسئله **طبقه‌بندیِ باینری یا دودویی** است، چون مجموعهٔ پاسخ‌های ممکن دقیقاً دو کلاس دارد. «باینری بودن» به تعدادِ کلاس‌ها مربوط است. «دوبعدی بودنِ فضای ویژگی» به تعدادِ ویژگی‌های ورودی مربوط است. در این مثال هر دو تعداد برابرِ دو شده‌اند، اما دو مفهومِ متفاوت را می‌شمارند: اگر ویژگیِ سومی اضافه کنیم، فضا سه‌بعدی می‌شود و مسئله با همین دو برچسب همچنان باینری باقی می‌ماند.

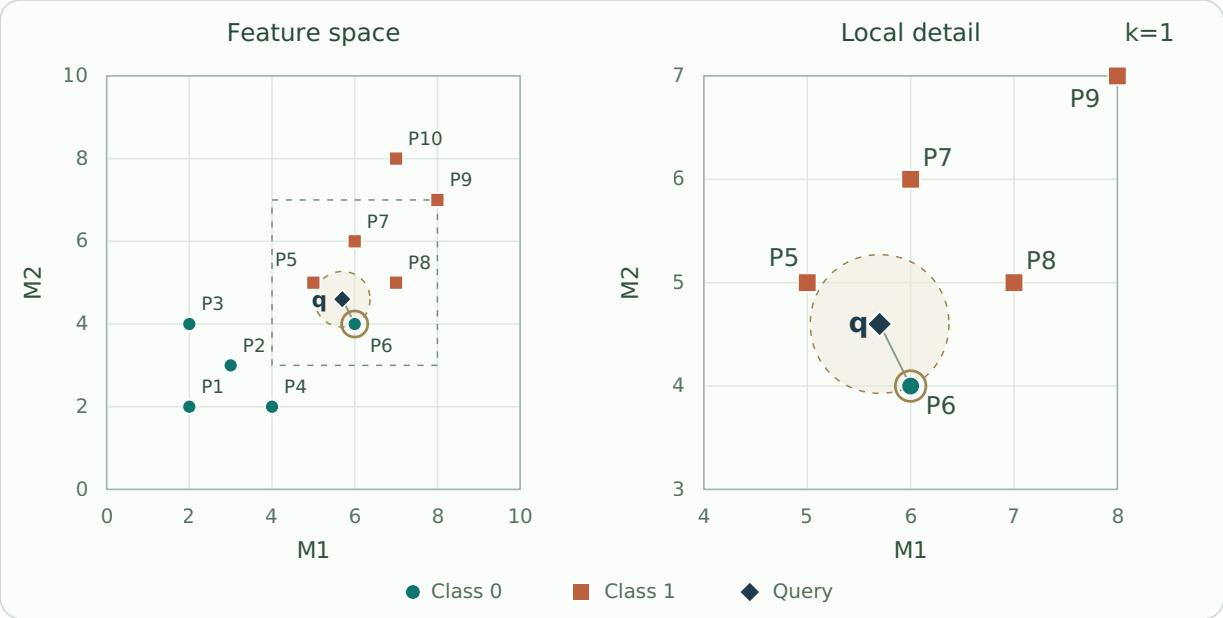
صفر و یک در اینجا کدِ دو دسته‌اند، نه اندازهٔ یک کمیت. برچسبِ ۱ به معنی یک واحد بیشتر بودنِ وضعیتِ S نیست و عددِ ۰٫۵ هم نامِ یک کلاس در قراردادِ ما نیست. خروجیِ طبقه‌بندی، انتخابِ یکی از دو کلاس است. در مقابل، اگر هدفِ مسئله تخمینِ یک مقدارِ عددی، مانند مقدارِ شاخصِ دیگری، بود، دربارهٔ رگرسیون صحبت می‌کردیم.

> هنگامِ محاسبهٔ فاصله، فقط دو مختصاتِ M1 و M2 را به کار می‌بریم. برچسبِ صفر یا یک، محورِ سومِ این نمودار و یکی از ورودی‌های فاصله نیست.

ابتدا از نزدیک‌ترین تجربه کمک بگیریم

دو امتیازِ مراجعه‌کنندهٔ تازه را با بردارِ پرس‌وجوی q نشان می‌دهیم. در شکل، این نمونه یک لوزیِ تیره است؛ هنوز برچسبِ واقعیِ آن را نمی‌دانیم. پیش از خواندنِ نتیجه، نزدیک‌ترین نقطه را در نمای بزرگ‌شده پیدا کنید و برچسبِ آن را بخوانید.

$$\mathrm{q} = (5.7,\ 4.6)$$



چپ: تمامِ پرونده‌ها؛ راست: بزرگ‌نماییِ محدودهٔ پرس‌وجو. در هر نما مقیاسِ دو محور برابر است؛ خط‌چین تا مرکزِ همسایهٔ انتخاب‌شده می‌رسد.

نزدیک‌ترین پرونده به q، نمونهٔ P6 است. برچسبِ ثبت‌شدهٔ آن صفر است. اگر فقط از همین یک تجربه کمک بگیریم، پیش‌بینیِ ما برای q نیز کلاسِ صفر خواهد بود. این همان ایدهٔ یک همسایهٔ نزدیک است: پاسخِ نزدیک‌ترین نمونه را به‌عنوانِ حدسِ خود پیشنهاد می‌کنیم.

در اینجا k برابرِ یک است؛ یعنی تعدادِ همسایه‌هایی که حقِ رأی دارند، یک مورد است. q را به جدولِ پرونده‌های قبلی اضافه نکرده‌ایم و برچسبِ فرضی به آن نداده‌ایم. فاصلهٔ نمونهٔ تازه را با نمونه‌هایی سنجیده‌ایم که پاسخِ آن‌ها از قبل معلوم بوده است.

> نزدیک‌ترین نقطه، یک شاهدِ محلی است. اینکه این شاهد برای پیش‌بینی کافی باشد، موضوعی است که باید با دادهٔ دارای پاسخ بررسی کنیم.

اگر سه تجربه را کنار هم بگذاریم چه می‌شود؟

مقدارِ k را از یک به سه تغییر دهید و q را ثابت نگه دارید. پیش از خواندنِ جدول، در نمای بزرگ‌شده سه همسایهٔ مشخص‌شده را ببینید و رأی‌های دو کلاس را بشمارید. تعدادِ همسایه‌ها عوض شده است؛ تعدادِ کلاس‌ها همان دو مورد است.



همان پرونده‌ها و همان q، با سه همسایه. نمای بزرگ‌شده، فاصلهٔ P7 از مرزِ انتخاب را خواناتر می‌کند؛ برای انتخاب، فاصلهٔ مرکزِ نقاط ملاک است.

برچسب	فاصلهٔ تقریبی تا q	پرونده	ترتیبِ نزدیکی
0	0.671	P6	1
1	0.806	P5	2
1	1.360	P8	3

اکنون به سه نزدیک‌ترین پرونده نگاه می‌کنیم. P6 به کلاسِ صفر تعلق دارد، اما P5 و P8 برچسبِ یک دارند. با یک رأیِ برابر برای هر همسایه، کلاسِ صفر یک رأی و کلاسِ یک دو رأی می‌گیرد. بنابراین پیش‌بینیِ سه‌همسایه‌ای، کلاسِ یک است؛ پاسخ با حالتِ تک‌همسایه‌ای فرق کرده است.

این روش، **k نزدیک‌ترین همسایه یا kNN** نام دارد. برای یک ورودیِ تازه، فاصله تا نمونه‌های قبلی را محاسبه می‌کنیم، k نمونهٔ نزدیک‌تر را برمی‌گزینیم و در طبقه‌بندیِ ساده، کلاسی را پیش‌بینی می‌کنیم که رأیِ بیشتری دارد. در این جزوه وزنِ رأیِ همسایه‌ها برابر است؛ نزدیک‌تر بودن، ترتیبِ انتخاب را تعیین می‌کند.

k تعدادِ کلاس‌ها نیست؛ تعدادِ همسایه‌های انتخاب‌شده است. در هر دو حالتِ k=1 و k=3، مسئله همچنان دو کلاس دارد. استفاده از چند همسایه می‌تواند اثرِ یک نمونه را کمتر کند، اما بزرگ‌تر کردنِ k تضمینِ بهتر شدنِ نتیجه نیست. هنوز نمی‌دانیم برچسبِ واقعیِ q چیست و از روی این دو حدس نمی‌توانیم یکی را درست اعلام کنیم.

از شکل به آرایه‌های NumPy برسیم

اکنون همان پرونده‌ها را واردِ NumPy می‌کنیم. شناسه‌ها را برای گزارش نگه می‌داریم، دو ویژگی را در X قرار می‌دهیم و برچسب‌ها را در y می‌گذاریم. ترتیبِ دو ویژگی در همهٔ سطرها و در پرس‌وجو ثابت است: ابتدا M1 و سپس M2.

در نسخهٔ سادهٔ kNN، مرحلهٔ آموزش شاملِ نگه‌داشتنِ نمونه‌های دارای پاسخ است. هنگامِ پیش‌بینی، ورودیِ تازه با همین نمونه‌ها مقایسه می‌شود. پس مدلِ ما علاوه بر k و معیارِ فاصله، به X و y ثبت‌شده نیاز دارد. این روش برای ساختنِ پیش‌بینی، خط یا معادله‌ای با ضرایبِ آموخته‌شده پیدا نمی‌کند.

```
import numpy as np

names = np.array([
    "P1", "P2", "P3", "P4", "P5",
    "P6", "P7", "P8", "P9", "P10"
])

X = np.array([
    [2, 2], [3, 3], [2, 4], [4, 2], [5, 5],
    [6, 4], [6, 6], [7, 5], [8, 7], [7, 8],
], dtype=float)

y = np.array([0, 0, 0, 0, 1, 0, 1, 1, 1, 1])
query = np.array([5.7, 4.6], dtype=float)

X.shape      # (10, 2)
y.shape      # (10,)
query.shape  # (2,)
```

برای نمونهٔ P6، سطرِ `X[5]` دو ویژگیِ ۶ و ۴ را نگه می‌دارد و `y[5]` برچسبِ صفرِ همان نمونه است. هماهنگیِ اندیس‌ها، اتصالِ «شرایطِ تجربه» به «پاسخِ تجربه» را حفظ می‌کند. اگر فقط یکی از دو آرایه را جابه‌جا کنیم، این اتصال خراب می‌شود.

X ده نمونه با دو ویژگی دارد؛ y ده پاسخِ متناظر را نگه می‌دارد. پرس‌وجو فقط دو ویژگی دارد. به آن مؤلفهٔ سومی برای برچسب اضافه نمی‌کنیم، زیرا همان برچسب را می‌خواهیم پیش‌بینی کنیم. همچنین نامِ P6 یا P10 واردِ محاسبهٔ فاصله نمی‌شود.

> شکلِ X تعدادِ سطرها و ویژگی‌ها را توصیف می‌کند. وجودِ فقط دو مقدارِ متفاوت در y، باینری بودنِ مسئله را مشخص می‌کند.

همسایه‌ها را پیدا کنیم، سپس رأی‌ها را بشماریم

پیش از اجرای کد، شکلِ خروجیِ `X - query` و تعدادِ فاصله‌های نهایی را پیش‌بینی کنید. فاصلهٔ دوبعدی همان رابطهٔ جلسهٔ قبل است؛ این بار هر سطر دو مؤلفه دارد:

$$d(\mathbf{x}, \mathbf{q}) = \sqrt{(x_1 - q_1)^2 + (x_2 - q_2)^2}$$

```
differences = X - query
squared = differences ** 2
sum_squared = np.sum(squared, axis=1)
distances = np.sqrt(sum_squared)  # shape: (10,)
```

در مرحلهٔ بعد، از اندیسِ فاصله‌های مرتب‌شده استفاده می‌کنیم تا هم شناسهٔ پرونده‌ها و هم برچسبِ آن‌ها با همان ترتیب انتخاب شوند:

```
k = 3
order = np.argsort(distances, kind="stable")
neighbors = order[:k]

names[neighbors]  # ['P6', 'P5', 'P8']
neighbor_labels = y[neighbors]  # [0, 1, 1]
```

اکنون مسئلهٔ رأی‌گیری، فقط شمارشِ دو برچسب است. مقایسهٔ بولی برای هر همسایه مشخص می‌کند آیا به یک کلاس تعلق دارد؛ جمعِ آن، تعدادِ رأی‌های همان کلاس را می‌دهد:

```
votes_0 = np.sum(neighbor_labels == 0)  # 1
votes_1 = np.sum(neighbor_labels == 1)  # 2

if votes_1 > votes_0:
    prediction = 1
else:
    prediction = 0

prediction  # 1
```

در این مثال، k=3 و وزنِ رأی‌ها برابر است؛ فرد بودنِ k مانعِ تساویِ تعدادِ رأی‌های دو کلاس می‌شود. تساویِ فاصله، مسئلهٔ دیگری است: اگر دو نمونه در مرزِ انتخاب هم‌فاصله باشند، `kind="stable"` موردِ زودتر در داده را مقدم می‌کند. تغییرِ ترتیبِ سطرها در چنین مرزی ممکن است انتخاب را عوض کند؛ این فقط قراردادِ ما برای رفعِ تساوی است.

نوبت شما: از مشورت تا یک پیش‌بینیِ قابلِ توضیح

برای هر بخش، علاوه بر نتیجهٔ محاسبه، توضیح دهید کدام اطلاعات در تصمیم دخالت کرده‌اند. جوابِ تمرینِ تازه در این جزوه نوشته نشده است.

1. **دو «دو» را از هم جدا کنید.** توضیح دهید چرا مسئلهٔ ما باینری و فضای ویژگیِ آن دوبعدی است. اگر تعدادِ شاخص‌ها پنج شود و برچسب‌ها همچنان صفر و یک بمانند، کدام ویژگیِ مسئله تغییر می‌کند؟
2. **پیش از اجرا، روی کاغذ پیش‌بینی کنید.** جایِ `q_new = np.array([4.6, 4.4])` را روی صفحه علامت بزنید. برای k برابرِ ۱، ۳ و ۵، همسایه‌ها و کلاسِ پیشنهادی را حدس بزنید؛ سپس با کد بررسی کنید و دلیلِ هر اختلاف را توضیح دهید.
3. **معنای کدِ برچسب را بررسی کنید.** فقط کدهای صفر و یک را در y با هم عوض کنید و مختصات و ترتیبِ سطرها را ثابت نگه دارید. همسایه‌ها، کدِ پیش‌بینی و معنای آن را قبل و بعد از این تغییر مقایسه کنید.
4. **مرزِ تمثیل را بشناسید.** در مشورت گرفتن، همهٔ افراد ممکن است تجربهٔ مرتبط نداشته باشند. توضیح دهید این نکته در kNN به انتخابِ ویژگی‌ها و تعریفِ فاصله چه ارتباطی دارد.
5. **پیش‌بینی را با واقعیت یکی نگیرید.** مدل برای پرس‌وجوی اصلی، با k=1 کلاسِ صفر و با k=3 کلاسِ یک را پیشنهاد داد. برای فهمیدنِ اینکه کدام پیش‌بینی درست بوده، به چه اطلاعاتی نیاز داریم؟ آیا هم‌رأی بودنِ تمامِ همسایه‌ها به‌تنهایی درستیِ پاسخ را ثابت می‌کند؟

k	همسایه‌ها به ترتیب	رأی ۰ / رأی ۱	پیش‌بینی
1	…	…	…
3	…	…	…
5	…	…	…

جدول را برای پرس‌وجوی تازه کامل کنید و نتیجهٔ کد را با حدسِ اولیه‌تان مقایسه کنید. در نسخهٔ وب، پس از جابه‌جاییِ q ابتدا رأی‌ها را خودتان بشمارید و سپس دکمهٔ نمایشِ نتیجه را بزنید. سهمِ رأیِ یک کلاس، به‌تنهایی احتمالِ واقعیِ وضعیتِ S را مشخص نمی‌کند.

این ده پرونده برای دیدنِ سازوکارِ روش‌اند. برای ارزیابیِ یک مدل، به نمونه‌های دارای پاسخ نیاز داریم که پاسخِ آن‌ها هنگامِ ساختنِ پیش‌بینی استفاده نشده باشد. در یادداشتِ بعد، همین نیاز را به تقسیمِ داده به آموزش و آزمون وصل می‌کنیم. مقیاس‌بندیِ مناسبِ ویژگی‌ها نیز بخشی از همان مسیر خواهد بود؛ هم‌مقیاس بودنِ دو شاخصِ این مثال، ویژگیِ دادهٔ ساختگیِ ماست.

برای مراجعهٔ بیشتر، توضیحِ مفهومیِ همسایه‌های نزدیک و مستنداتِ ابزارهای NumPy:

Nearest neighbors · Conceptual guide

NumPy · argsort

Google · Supervised learning